

JavaScript INTERVIEW Questions

1. JavaScript Fundamentals

Q1. What is JavaScript and where is it used?

JavaScript is a high-level, interpreted programming language primarily used for building interactive web applications. It runs in the browser as well as on the server using Node.js. It is widely used for frontend development, backend services, and even data processing tasks.

Q2. What are primitive and non-primitive data types?

Primitive data types include number, string, boolean, null, undefined, bigint, and symbol. These are immutable and stored by value. Non-primitive types such as objects and arrays are mutable and stored by reference.

Q3. What is the difference between `var`, `let`, and `const`?

`var` is function-scoped and can be redeclared and updated. `let` is block-scoped and can be updated but not redeclared. `const` is also block-scoped but cannot be reassigned after initialization. In modern JavaScript, `let` and `const` are preferred for better scope control.

2. Execution & Scope

Q4. What is hoisting?

Hoisting is JavaScript's default behavior of moving declarations to the top of their scope before execution. Variables declared with `var` are initialized with undefined, while `let` and `const` are hoisted but not initialized, leading to a temporal dead zone.

Q5. What is scope in JavaScript?

Scope determines the accessibility of variables. JavaScript has global scope, function scope, and block scope. Variables declared inside a function are not accessible outside it.

Q6. What is closure? (VERY IMPORTANT)

A closure is a function that retains access to variables from its lexical scope even after the outer function has finished execution. Closures are widely used in callbacks, event handlers, and data privacy patterns.

3. Functions & Concepts

Q7. What is the difference between function declaration and function expression?

Function declarations are hoisted and can be called before definition, while function expressions are not hoisted and must be defined before use.

Q8. What are arrow functions?

Arrow functions provide a shorter syntax and do not have their own `this`. Instead, they inherit `this` from the surrounding context.

4. Objects & Arrays

Q9. How are objects used in JavaScript?

Objects store key-value pairs and are used to represent real-world entities. They can contain properties and methods.

Q10. Difference between shallow copy and deep copy?

A shallow copy copies references of nested objects, while a deep copy creates a completely independent copy of the object.

5. Asynchronous JavaScript (VERY IMPORTANT for O9)

Q11. What is asynchronous programming?

Asynchronous programming allows tasks to run independently without blocking execution. It is essential for handling APIs and long-running tasks.

Q12. What is a Promise?

A Promise represents the eventual completion or failure of an asynchronous operation. It has three states: pending, fulfilled, and rejected.

Q13. What is async/await?

`async/await` is syntactic sugar over Promises, making asynchronous code look synchronous and easier to read.

Q14. What is the event loop? (VERY IMPORTANT)

The event loop is a mechanism that handles asynchronous operations in JavaScript. It continuously checks the call stack and task queue to execute code without blocking the main thread.

6. DOM & Browser Concepts

Q15. What is DOM?

The Document Object Model represents the structure of a web page as a tree of objects that JavaScript can manipulate.

Q16. What is event delegation?

Event delegation is a technique where a parent element handles events for its child elements using event bubbling.

7. ES6+ Features

Q17. What are template literals?

Template literals allow embedding expressions in strings using backticks and ``${}`` syntax.

Q18. What is destructuring?

Destructuring allows unpacking values from arrays or objects into variables.

Q19. What is spread and rest operator?

Spread expands elements, while rest collects multiple elements into one.

8. Arrays & Methods

Q20. Difference between map, filter, and reduce?

`map` transforms elements, `filter` selects elements based on condition, and `reduce` aggregates values into a single result.

9. Prototypes & Inheritance

Q21. What is prototype in JavaScript?

Every JavaScript object has a prototype, which is another object from which it inherits properties.

Q22. What is prototypal inheritance?

It is a mechanism where objects inherit properties from other objects via the prototype chain.

10. Memory & Performance

Q23. What is garbage collection?

JavaScript automatically frees memory by removing objects that are no longer reachable.

11. Error Handling

Q24. How do you handle errors in JavaScript?

Errors are handled using try-catch blocks, and custom errors can be thrown using the throw keyword.

12. Real O9-Level Scenario Questions

Q25. How would you handle multiple API calls efficiently?

I would use Promise.all to run multiple API calls in parallel and handle results together. If one fails, error handling strategies like Promise.allSettled can be used.

Q26. How do you optimize a slow web page?

Optimization includes minimizing DOM manipulation, lazy loading, caching API responses, and reducing unnecessary re-renders.

Q27. How would you manage state in a large application?

State can be managed using tools like Redux or context APIs, ensuring centralized and predictable data flow.

13. Coding-Oriented Questions

Q28. Reverse a string

```
function reverseString(str) {  
  return str.split("").reverse().join("");  
}
```

Q29. Check palindrome

```
function isPalindrome(str) {  
  return str === str.split("").reverse().join("");  
}
```

Q30. Debounce function (IMPORTANT)

```
function debounce(fn, delay) {  
  let timer;  
  return function(...args) {  
    clearTimeout(timer);  
    timer = setTimeout(() => fn.apply(this, args), delay);  
  }  
}
```

14. Advanced Concepts

Q31. What is currying?

Currying is a technique of converting a function with multiple arguments into a sequence of functions each taking one argument.

Q32. What is memoization?

Memoization is an optimization technique where results of expensive function calls are cached.

15. System Thinking Questions (VERY IMPORTANT)

Q33. How would you design a real-time dashboard?

I would use WebSockets for real-time updates, efficient state management, and optimized rendering strategies.

Q34. How would you handle large datasets in frontend?

I would use pagination, lazy loading, and virtualization to improve performance.

16. Tricky Questions

Q35. Difference between == and ===?

== compares values after type coercion, while === compares both value and type.

Q36. What is NaN?

NaN represents “Not a Number” and is returned for invalid numerical operations.

17. Final High-Level Questions

Q37. What is modular JavaScript?

Modular JavaScript divides code into smaller reusable modules.

Q38. What is tree shaking?

Tree shaking removes unused code during bundling.

18. Last Important Questions

Q39. What is REST API?

REST API is a way of communication between client and server using HTTP methods.

Q40. What is CORS?

CORS is a security mechanism that controls access to resources from different origins.

WEB DEV CODING QUESTIONS

1–10: DOM Manipulation

Q1. Change text of an element

```
function changeText() {  
  document.getElementById("demo").innerText = "Hello World";  
}
```

Q2. Change CSS style dynamically

```
function changeColor() {  
  document.getElementById("box").style.backgroundColor = "blue";  
}
```

Q3. Add a new element to DOM

```
function addItem() {  
  let li = document.createElement("li");  
  li.innerText = "New Item";  
  document.getElementById("list").appendChild(li);  
}
```

Q4. Remove an element

```
function removeItem() {  
  let el = document.getElementById("item");  
  el.parentNode.removeChild(el);  
}
```

Q5. Get input value

```
function getInput() {  
  let value = document.getElementById("inputBox").value;  
  console.log(value);  
}
```

Q6. Toggle class

```
function toggleClass() {  
  document.getElementById("box").classList.toggle("active");  
}
```

Q7. Select multiple elements

```
function highlightAll() {  
  let items = document.querySelectorAll(".item");  
  items.forEach(el => el.style.color = "red");  
}
```

Q8. Event listener

```
document.getElementById("btn").addEventListener("click", function() {  
  alert("Button clicked");  
});
```

Q9. Prevent default form submission

```
document.getElementById("form").addEventListener("submit", function(e) {  
  e.preventDefault();  
  console.log("Form prevented");  
});
```

Q10. Event delegation

```
document.getElementById("list").addEventListener("click", function(e) {  
  if (e.target.tagName === "LI") {  
    console.log(e.target.innerText);  
  }  
});
```

11–20: Forms & Validation

Q11. Validate empty input

```
function validate() {
```

```
let val = document.getElementById("name").value;
if (val === "") {
    alert("Field cannot be empty");
}
}
```

Q12. Email validation

```
function validateEmail(email) {
    return /^[^s@]+@[^s@]+\.[^s@]+$/.test(email);
}
```

Q13. Password strength check

```
function strongPassword(pwd) {
    return pwd.length >= 8;
}
```

Q14. Show/hide password

```
function togglePassword() {
    let input = document.getElementById("pwd");
    input.type = input.type === "password" ? "text" : "password";
}
```

Q15. Count characters in textarea

```
function countChars() {
    let text = document.getElementById("msg").value;
    document.getElementById("count").innerText = text.length;
}
```

Q16. Auto-fill form

```
function fillForm() {
    document.getElementById("name").value = "John";
}
```

Q17. Disable button after click

```
function disableBtn(btn) {  
  btn.disabled = true;  
}
```

Q18. Reset form

```
function resetForm() {  
  document.getElementById("form").reset();  
}
```

Q19. Real-time validation

```
document.getElementById("name").addEventListener("input", function() {  
  if (this.value.length < 3) {  
    this.style.border = "2px solid red";  
  } else {  
    this.style.border = "2px solid green";  
  }  
});
```

Q20. Dropdown value

```
function getSelected() {  
  let val = document.getElementById("select").value;  
  console.log(val);  
}
```

21–30: Async & API Handling (VERY IMPORTANT)

Q21. Fetch API data

```
async function fetchData() {  
  let res = await fetch("https://api.example.com/data");  
  let data = await res.json();  
  console.log(data);  
}
```

Q22. POST request

```
async function postData() {  
  let res = await fetch("https://api.example.com", {  
    method: "POST",  
    body: JSON.stringify({name: "John"}),  
    headers: {"Content-Type": "application/json"}  
  });  
}
```

Q23. Handle API error

```
async function getData() {  
  try {  
    let res = await fetch("url");  
    let data = await res.json();  
  } catch (err) {  
    console.log("Error:", err);  
  }  
}
```

Q24. Loading spinner

```
async function loadData() {  
  document.getElementById("loader").style.display = "block";  
  await fetchData();  
  document.getElementById("loader").style.display = "none";  
}
```

Q25. Debounce input search

```
function debounce(fn, delay) {  
  let timer;  
  return function(...args) {  
    clearTimeout(timer);  
    timer = setTimeout(() => fn(...args), delay);  
  };  
}
```

Q26. Infinite scroll

```
window.addEventListener("scroll", () => {
```

```
    if (window.innerHeight + window.scrollY >= document.body.offsetHeight) {  
        console.log("Load more data");  
    }  
});
```

Q27. Retry API

```
async function retry(fn, retries) {  
    try {  
        return await fn();  
    } catch {  
        if (retries === 0) throw "Failed";  
        return retry(fn, retries - 1);  
    }  
}
```

Q28. Parallel API calls

```
async function parallel() {  
    let [a, b] = await Promise.all([fetch("url1"), fetch("url2")]);  
}
```

Q29. Sequential API calls

```
async function sequential() {  
    let a = await fetch("url1");  
    let b = await fetch("url2");  
}
```

Q30. Cache API response

```
let cache = {};  
  
async function cachedFetch(url) {  
    if (cache[url]) return cache[url];  
    let res = await fetch(url);  
    let data = await res.json();  
    cache[url] = data;  
    return data;  
}
```

31–40: Browser APIs & UI Logic

Q31. LocalStorage save

```
localStorage.setItem("name", "John");
```

Q32. LocalStorage get

```
let name = localStorage.getItem("name");
```

Q33. Dark mode toggle

```
function toggleDark() {  
  document.body.classList.toggle("dark");  
}
```

Q34. Copy to clipboard

```
function copyText() {  
  navigator.clipboard.writeText("Hello");  
}
```

Q35. Detect online/offline

```
window.addEventListener("online", () => console.log("Online"));  
window.addEventListener("offline", () => console.log("Offline"));
```

Q36. Drag and drop

```
function allowDrop(e) { e.preventDefault(); }  
function drop(e) {  
  e.preventDefault();  
  let data = e.dataTransfer.getData("text");  
  e.target.appendChild(document.getElementById(data));  
}
```

Q37. Set timeout

```
setTimeout(() => console.log("Hello"), 2000);
```

Q38. Set interval

```
setInterval(() => console.log("Running"), 1000);
```

Q39. Clear interval

```
let id = setInterval(() => {}, 1000);  
clearInterval(id);
```

Q40. Scroll to top

```
function scrollTop() {  
  window.scrollTo({top: 0, behavior: "smooth"});  
}
```